

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені ІВАНА ФРАНКА
Факультет прикладної математики та інформатики

Операційні системи та системне програмування

ЛАБОРАТОРНА РОБОТА №2

Виконала:

Ст. Макух Вікторія

Група: ПМі-46

Постановка задачі

1. Два різні текстові файли мають розширення імені *.dat. Повні імена файлів невідомі. У файлах записані числові дані як цілі або дійсні числа, необхідні для деякої задачі (даних є не менше 20 елементів в кожному файлі). Окремі елементи даних можуть бути синтаксично неправильні з точки зору програми, яка має їх прочитати – за правилами визначення цілих і дійсних чисел.

2. Лише відомо, що обидва файли будуть разом в поточній папці програми або у вкладеній папці problem. Попередньої інформації про конкретне місце розташування файлів немає. Вважати, що інших файлів з розширенням імені *.dat нема, але можуть бути файли з іншими розширеннями імен.

3. Програма має в формі діалогу (віконного чи консольного) забезпечити вибір і виконання перерахованих далі операцій з можливістю фіксування протоколу роботи в текстовому файлі.

Виконати такі операції:

- 1) Встановити і показати повні імена файлів з розширенням імені *.dat.
- 2) Знайти і показати повний шлях від кореня логічного диску до файлу.
- 3) Отримати і показати в формі таблиці всі параметри/властивості і атрибути знайдених файлів, які можна отримати програмно, зокрема повне ім'я, розмір і дату створення (редагування);

Опис функціональності:

1. Пошук і відображення файлів

- При запуску програми автоматично сканується поточний каталог і підкаталог problem на наявність файлів із розширенням .dat.
- Програма заповнює список (у вигляді ListBox), де відображаються тільки імена знайдених файлів.
- Ведеться журнал (Logger), де фіксується кількість знайдених файлів.

2. Вибір файлу

- Користувач може вибрати будь-який файл із наданого списку (ListBox).
- При виборі файлу програма виконує наступні дії:
 - Вмикає та відображає вкладки, призначені для роботи з вибраним файлом.

- Виводить властивості вибраного файлу (наприклад, розмір, розширення, дата створення, модифікації, доступу, атрибути).
- Завантажує і відображає вміст файлу у вкладці "Content".
- Очищає поля для результатів обчислень (щоб уникнути змішування із результатами попереднього файлу).
- Ведеться лог події вибору користувачем файлу.

3. Відображення властивостей файлу

- Для вибраного файлу у вкладці "Properties" відображаються детальні властивості:
 - Шлях до файлу (повний шлях у файловій системі),
 - Розмір файлу,
 - Розширення файлу,
 - Дата створення файлу,
 - Дата останньої модифікації файлу,
 - Дата останнього доступу до файлу,
 - Атрибути файлу
- Якщо файл не має специфічних атрибутів, відображається відповідний напис "None".

4. Відображення вмісту файлу

- У вкладці "Content" відображається весь текстовий вміст вибраного файлу.
- Вміст файлу зчитується в текстовий компонент (TextBox), щоб користувач міг переглянути його вміст.

5. Обчислення

- У вкладці "Calculation" користувач може виконати підрахунок суми числових значень з вмісту файлу.
- Процес обчислення виглядає так:
 - Текст із файлу розбивається на елементи, розділені пробілами.
 - Кожен елемент намагається конвертуватися в число (типу decimal).
 - Якщо конвертація елемента не вдається, замість нього використовується значення за замовчуванням, визначене в полі invalidReplacementValue (зазначене користувачем або програмою).

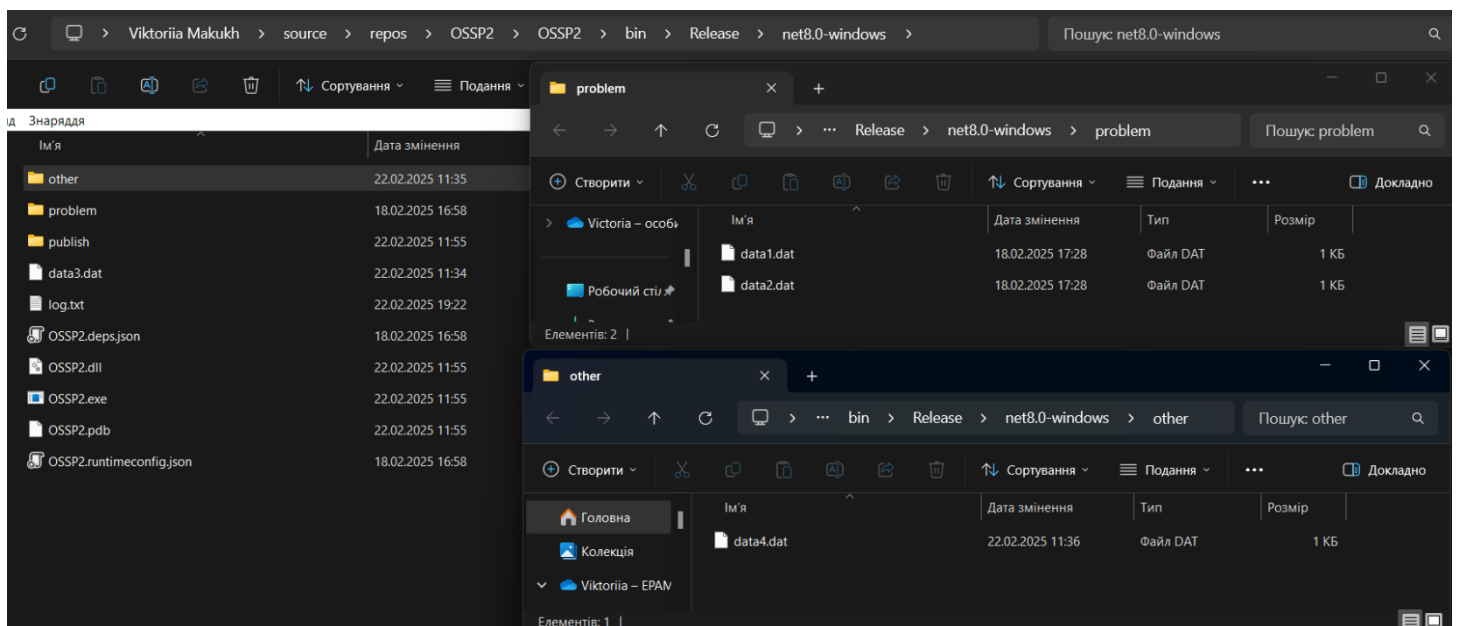
- Виводяться два результати:
 - Початковий вміст файлу (елементи, розділені символом +).
 - Перетворений вміст файлу (з урахуванням заміненних значень).
- Підраховується загальна сума всіх чисел і відображається у відповідному текстовому полі.
- У разі виникнення помилки (наприклад, файл пошкоджений або сформатований некоректно), користувач отримує повідомлення, а деталі помилки записуються до журналу.

6. Логування

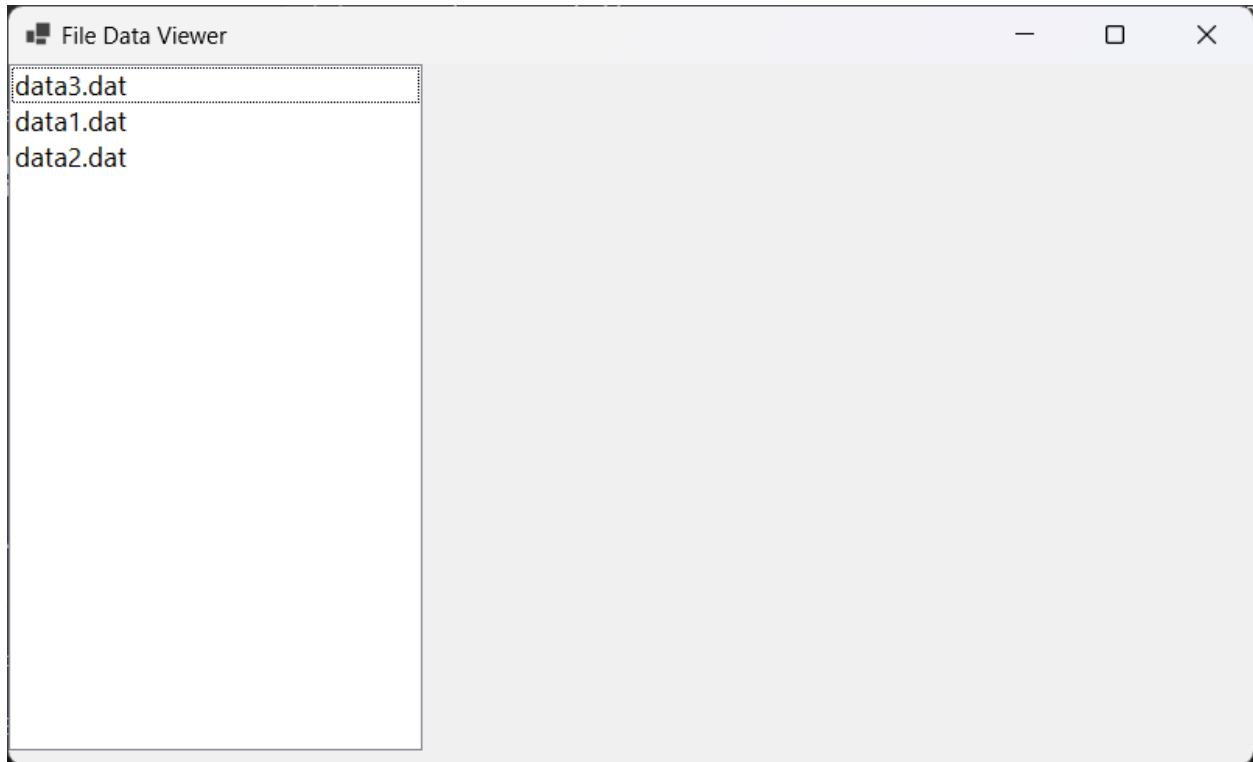
- Додаток використовує логування для запису важливих подій:
 - Запуск програми,
 - Виявлення файлів,
 - Вибір користувачем файлу,
 - Виконання обчислень (або помилок в процесі).
- Логи допомагають задокументувати дії користувача та спростити діагностику проблем.

Демонстрація роботи:

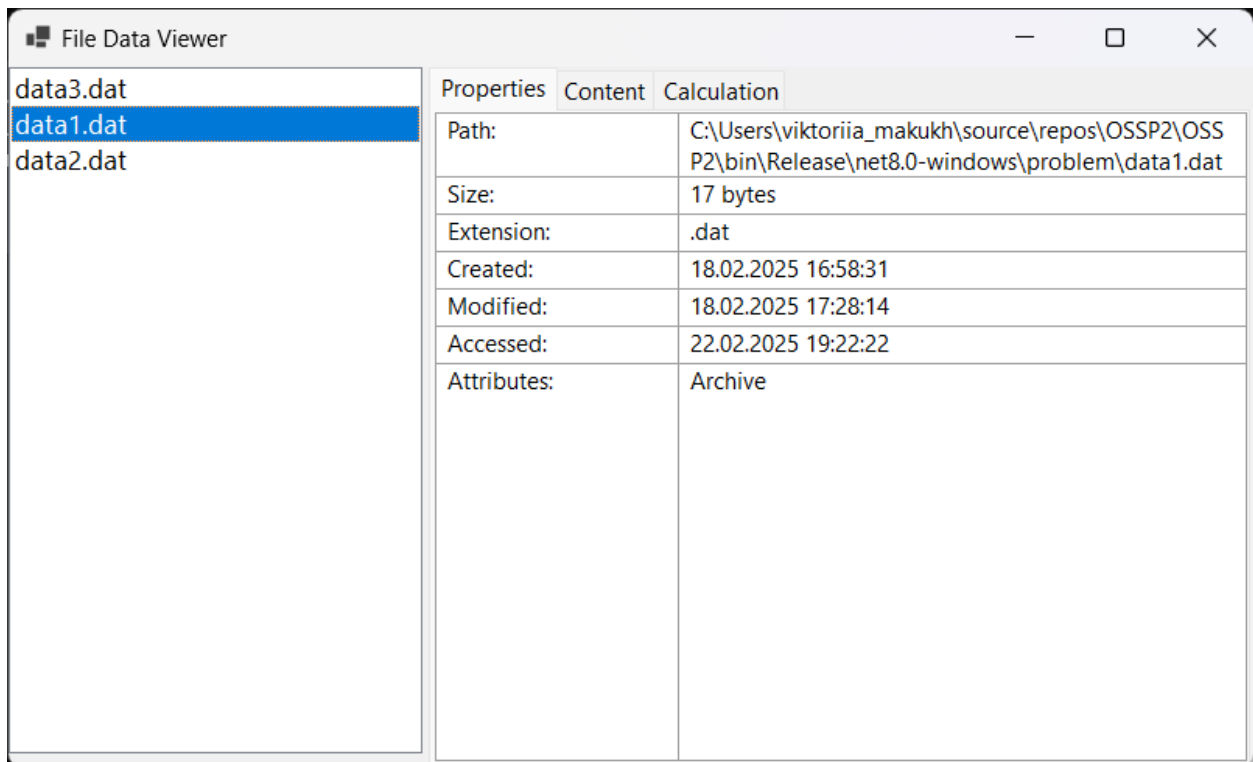
1. Перед запуском підготувала папку problem та папку other із файлами .dat. Також очистила файл з логами. Розташування файлів:
 - data1.dat, data2.dat - папка problem
 - data3.dat – у основній папці net8.0-windows
 - data4.dat – папка other



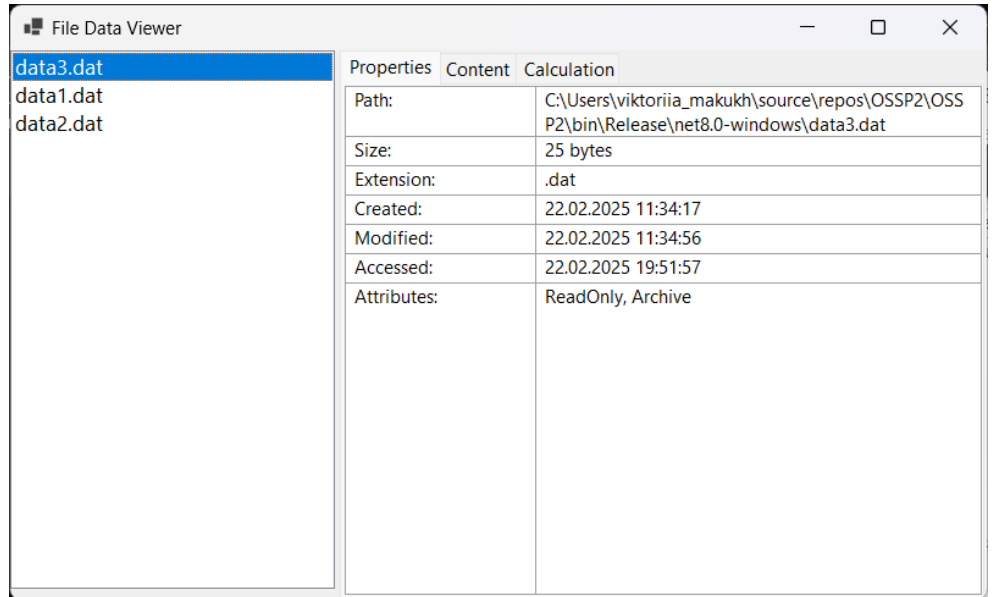
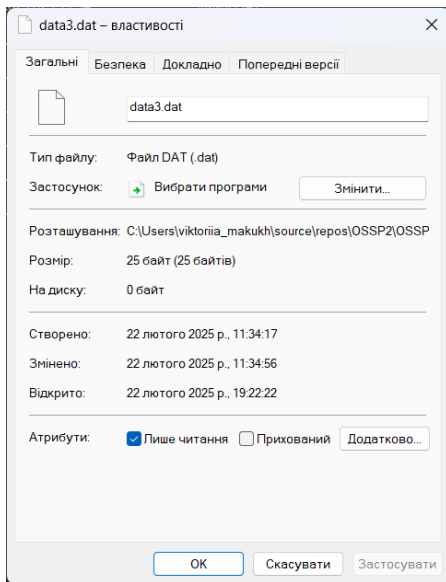
2. При запуску бачимо, що відобразились лише потрібні нам файли data1.dat, data2.dat, data3.dat



3. Виберемо перший файл і подивимось його властивості та атрибути

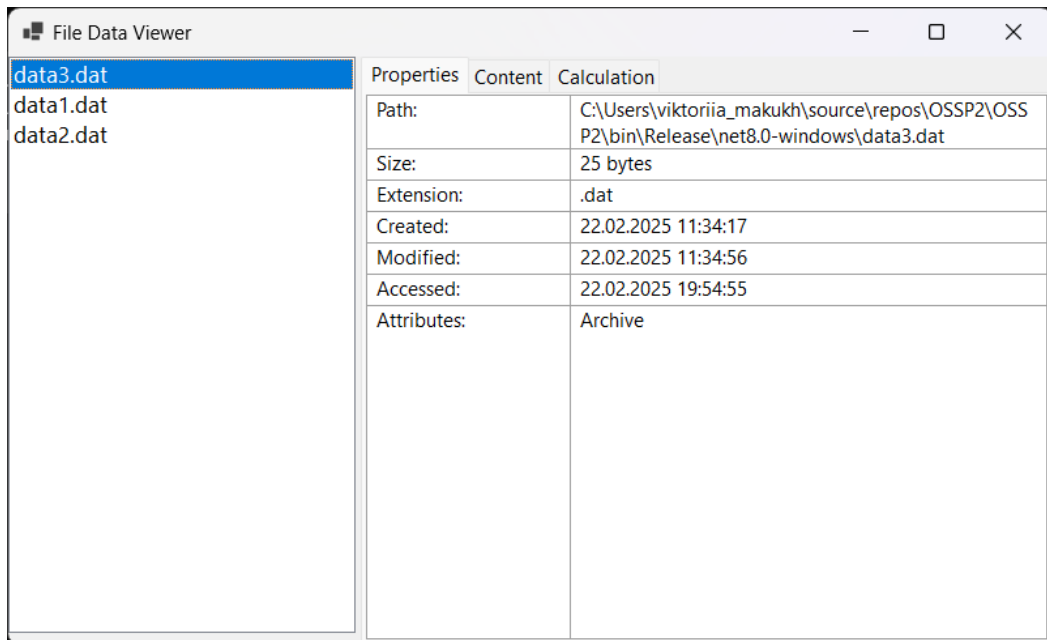
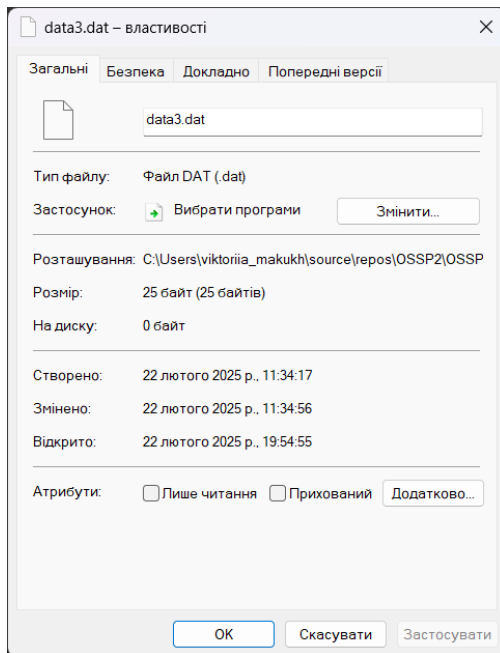


4. Тепер виберемо другий файл, який має атрибут Read-Only і подивимось його властивості та атрибути



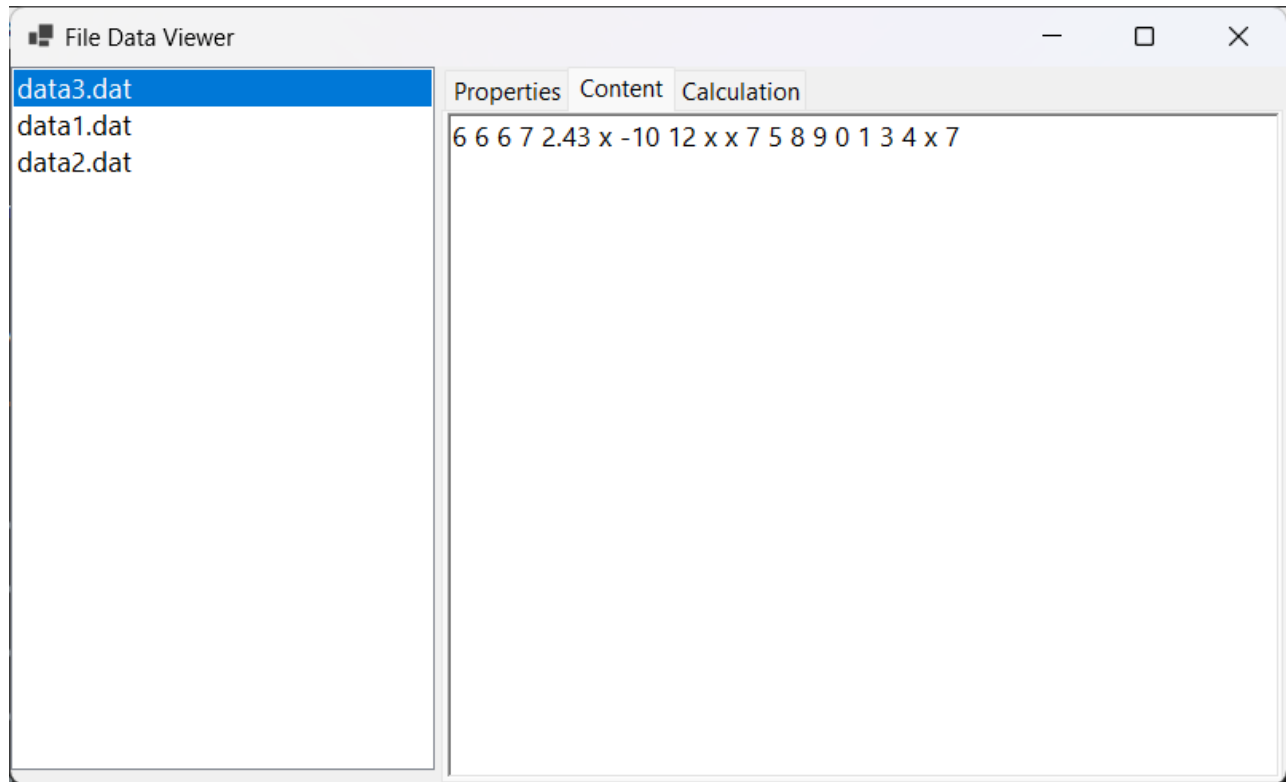
Як бачимо, програма відображає правильно атрибути

5. Спробуємо забрати цей атрибут і перевірити, чи зміниться це у програмі

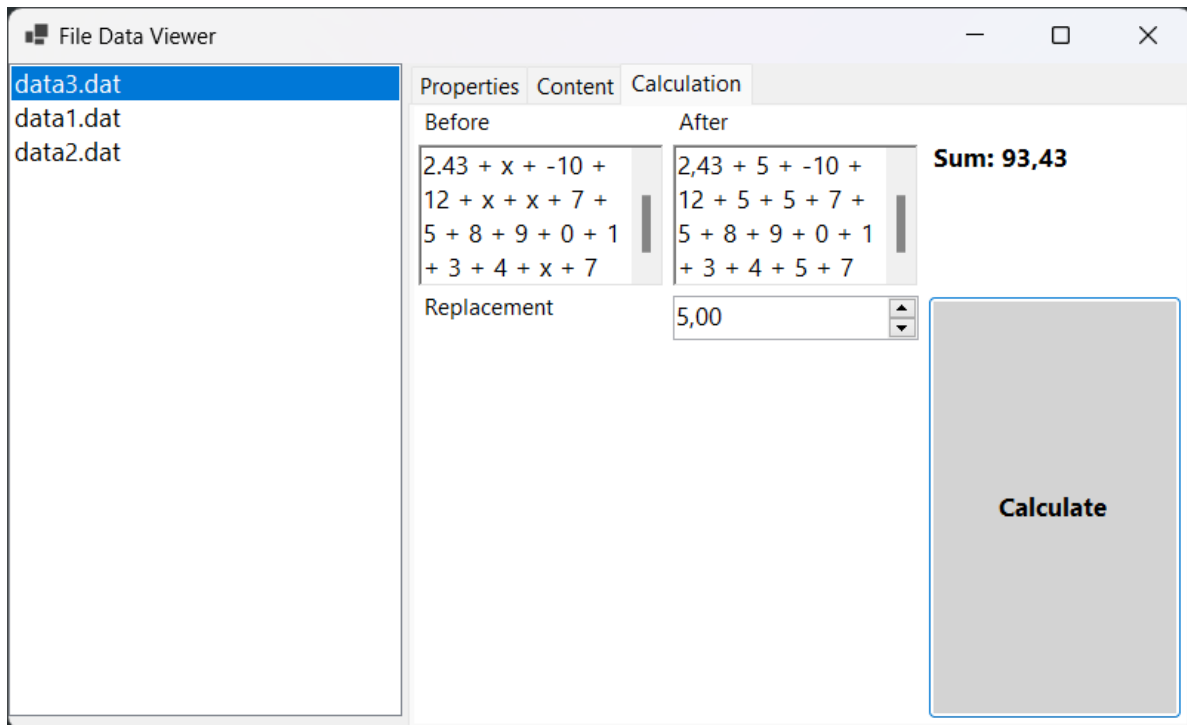


При повторному натиску програма відобразила цю зміну

6. Переглянемо вміст файлу

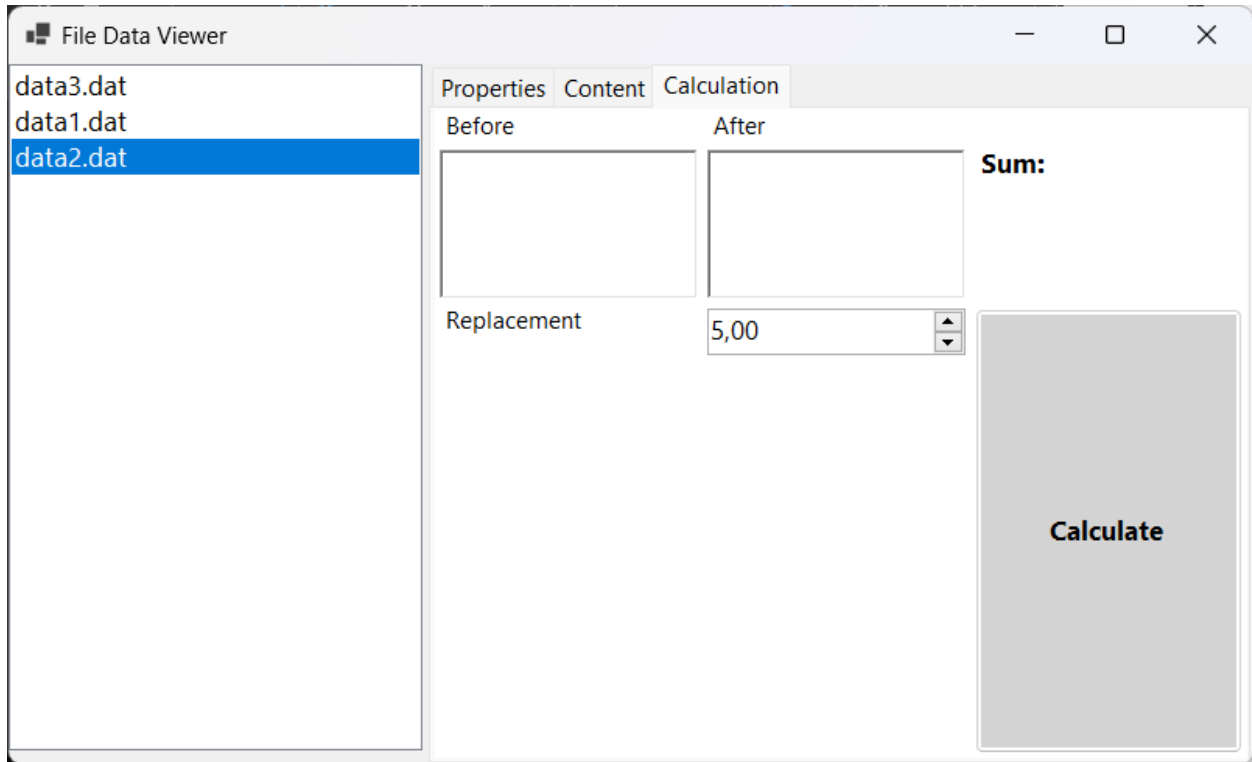


7. Спробуємо здійснити калькуляцію суми, замінюючи помилковий формат(x) на конкретне число



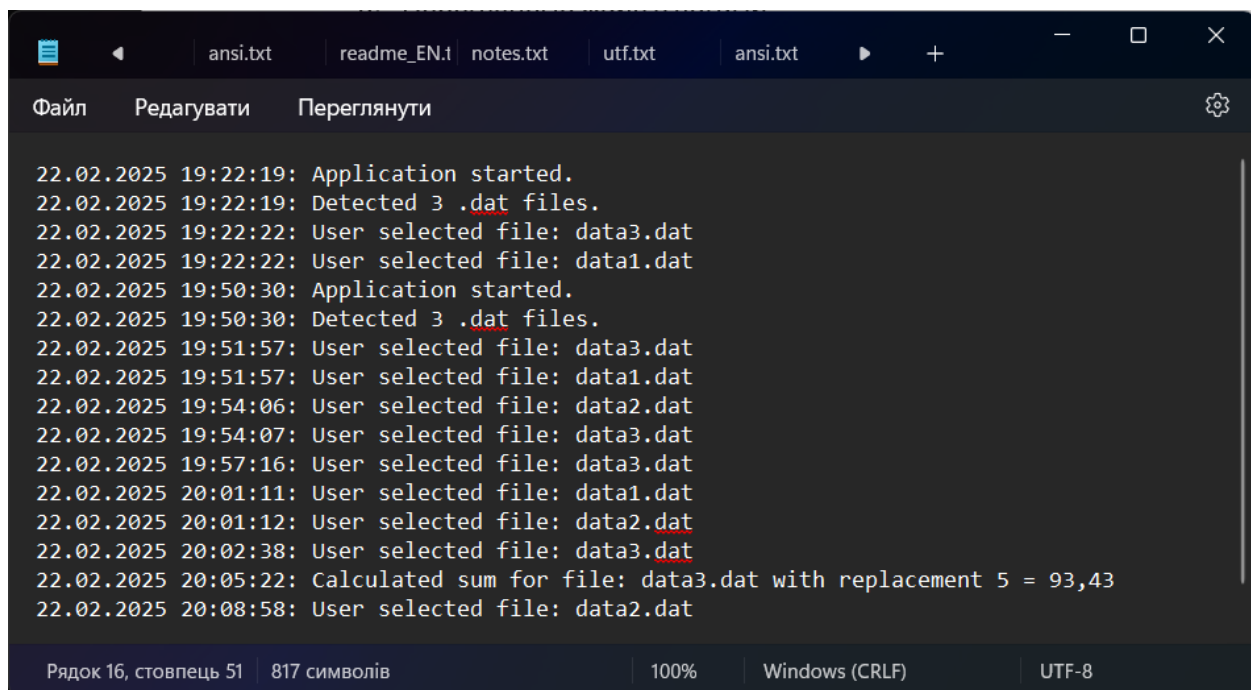
Як бачимо, програма замінила помилковий формат та правильно порахувала суму

8. Перейдемо ще на третій файл, аби переконатись, що програма без проблем переходить між файлами



Як бачимо, вікно очистилось

9. Переглянемо файл з логами



Алгоритм роботи програми

Загальний алгоритм програми:

1. Ініціалізація програми:

- Зчитуються файли із поточного каталогу та підкаталогу problem.
- Відображається список доступних файлів.

```
namespace OSSP2
{
    3 references
    public partial class Form1 : Form
    {
        private readonly Logger _logger;           // Логер для запису дій та подій
        private readonly string[] _filePaths;       // Масив шляхів до знайдених файлів
        private FileInfo? _currentFile;             // Поточний вибраний файл

        1 reference
        public Form1()
        {
            _logger = new Logger();                 // Ініціалізація об'єкту логера
            InitializeComponent();                   // Ініціалізація UI-компонентів

            _logger.Log("Application started.");    // Лог повідомлення про запуск програми

            // Визначення поточного каталогу і підкаталогу для пошуку файлів
            var currentDirectory = new DirectoryInfo(Directory.GetCurrentDirectory());
            var subDirectory = new DirectoryInfo(Path.Combine(Directory.GetCurrentDirectory(), "problem"));

            // Пошук всіх файлів з розширенням ".dat" у поточній директорії та піддиректорії
            var rootFiles = currentDirectory.GetFiles("*.dat");
            var subDirFiles = subDirectory.Exists ? subDirectory.GetFiles("*.dat") : Array.Empty<FileInfo>();

            // Об'єднання списків файлів з поточного каталогу і підкаталогу
            _filePaths = rootFiles.Concat(subDirFiles).Select(file => file.FullName).ToArray();

            // Додавання файлів у список для відображення в ListBox
            fileListBox.Items.AddRange(_filePaths.Select(Path.GetFileName).ToArray());
            _logger.Log($"Detected {_filePaths.Length} .dat files."); // Лог кількості знайдених файлів
        }
    }
}
```

2. Вибір файлу:

Коли користувач вибирає файл у ListBox, визначається його шлях до файлу у вигляді FileInfo об'єкта.

Якщо файл обраний:

- Активується відображення вкладок (tabs).
- Викликаються методи:
 - ShowFileProperties — для відображення властивостей файлу.
 - ShowFileContent — для відображення вмісту файлу.
 - ClearCalculationResults — для очищення результатів обчислення.
 - Логується, який файл був обраний.

```
// Обробка вибору файлу у ListBox
1 reference
private void fileListBox_SelectedIndexChanged(object sender, EventArgs e)
{
    // Якщо жоден файл не вибрано, вихід з методу
    if (fileListBox.SelectedIndex < 0)
        return;

    // Отримання інформації про вибраний файл
    _currentFile = new FileInfo(_filePaths[fileListBox.SelectedIndex]);

    if (_currentFile != null)
    {
        // Увімкнення відображення вкладок
        tabs.Enabled = true;
        tabs.Visible = true;

        // Відображення властивостей файлу та його вмісту
        ShowFileProperties(); // Відображення властивостей файлу
        ShowFileContent(); // Відображення вмісту файлу у вкладці "Content"
        ClearCalculationResults(); // Очищення результатів обчислень

        _logger.Log($"User selected file: {_currentFile.Name}"); // Лог події вибору файлу
    }
    else
    {
        // Вимкнення вкладок, якщо файл не вибрано
        tabs.Enabled = false;
        tabs.Visible = false;
    }
}
```

```
// Метод для відображення властивостей вибраного файлу
1 reference
private void ShowFileProperties()
{
    if (_currentFile == null)
        return;

    // Виведення основних властивостей файлу
    filePathLabel.Text = _currentFile.FullName; // Повний шлях до файлу
    fileSizeLabel.Text = $"{_currentFile.Length} bytes"; // Розмір файлу
    fileExtensionLabel.Text = _currentFile.Extension; // Розширення файлу
    fileCreatedLabel.Text = _currentFile.CreationTime.ToString(); // Дата створення
    fileModifiedLabel.Text = _currentFile.LastWriteTime.ToString(); // Дата останньої модифікації
    fileAccessedLabel.Text = _currentFile.LastAccessTime.ToString(); // Дата останнього доступу

    // Визначення атрибутів файлу
    var attributes = Enum.GetValues(typeof(FileAttributes))
        .Cast<FileAttributes>()
        .Where(attr => attr != FileAttributes.Normal && _currentFile.Attributes.HasFlag(attr) && attr != 0) // Виключення "Normal" і значення 0
        .ToList();

    // Відображення атрибутів
    fileAttributesLabel.Text = attributes.Any()
        ? string.Join(", ", attributes)
        : "None"; // Якщо атрибутів немає
}

// Метод для відображення вмісту файлу у текстовому полі
1 reference
private void ShowFileContent()
{
    if (_currentFile == null)
        return;

    // Зчитування і відображення тексту з файлу
    fileContentBox.Text = File.ReadAllText(_currentFile.FullName);
}
```

3. Обчислення:

Користувач натискає кнопку "Calculate"

Алгоритм дій методу, який обробляє цю подію:

- Перевіряє, чи було вибрано файл. Якщо не вибрано, показується повідомлення про помилку.
- Зчитує вміст файлу рядком.
- Розбиває текст на елементи (розділені пробілами).
- Для кожного елемента:
 - Якщо він містить число, це значення використовується.
 - Якщо не є числом, замінюється значенням за замовчуванням (`invalidReplacementValue.Value`).
- Оновлює текстові блоки для відображення:
 - Даних до обробки (елементи розділені символом `+`).
 - Даних після обробки (оброблені значення також розділені `+`).
- Обчислює суму числових значень.
- Виводить суму
- Логує результат обчислення.
- У разі помилки (наприклад, файл пошкоджений) показується повідомлення про помилку і вона записується до журналу.

```
// Обробка натискання кнопки "calculateButton" для підрахунку суми
1 reference
private void calculateButton_Click(object sender, EventArgs e)
{
    if (_currentFile == null)
    {
        // Якщо файл не вибраний, показати повідомлення про помилку
        MessageBox.Show("No file selected!", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        return;
    }

    try
    {
        // Зчитування вмісту файлу
        var content = File.ReadAllText(_currentFile.FullName);

        // Поділ вмісту на елементи, видаляючи зайві пробіли
        var elements = content.Split(' ', StringSplitOptions.RemoveEmptyEntries);

        // Обробка елементів: перетворення у числа або заміна на значення за замовчуванням
        var replacementValue = invalidReplacementValue.Value;
        var parsedResults = elements.Select(el =>
            decimal.TryParse(el, NumberStyles.Any, CultureInfo.InvariantCulture, out var num) ? num : replacementValue);

        // Оновлення текстових блоків зі значеннями до і після перетворення
        resultsBeforeBox.Text = string.Join(" + ", elements);
        resultsAfterBox.Text = string.Join(" + ", parsedResults);

        // Обчислення суми числових значень
        var totalSum = parsedResults.Sum();
        resultLabel.Text = $"Sum: {totalSum}";

        // Лог успішного обчислення
        _logger.Log($"Calculated sum for file: {_currentFile.Name} with replacement {replacementValue} = {totalSum}");
    }
    catch (Exception ex)
    {
        // Обробка помилки зчитування файлу
        MessageBox.Show($"Error reading file: {ex.Message}", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        _logger.Log($"Error calculating file: {ex.Message}");
    }
}
```

4. Логування:

- Усі ключові події програми (запуск, вибір файлу, обчислення або помилка) записуються в логи

```
namespace OSSP2
{
    // Клас Logger відповідає за ведення журналу подій програми
    3 references
    public class Logger
    {
        private readonly string _logFile; // Шлях до файлу для зберігання логів

        // Конструктор класу Logger
        1 reference
        public Logger()
        {
            // Визначення шляху до файлу логів у поточній директорії програми
            _logFile = Path.Combine(Directory.GetCurrentDirectory(), "log.txt");
        }

        // Метод Log записує повідомлення у файл логів з часовою міткою
        5 references
        public void Log(string message)
        {
            // Форматування повідомлення з додаванням дати та часу
            var timeStampedMessage = $"{DateTime.Now}: {message}{Environment.NewLine}";

            // Додавання повідомлення у файл логів (створює файл, якщо його ще немає)
            File.AppendAllText(_logFile, timeStampedMessage);
        }
    }
}
```

Висновок: У ході виконання цього завдання мені вдалося створити віконну програму на основі Windows Forms із використанням мови програмування C#. Програма успішно реалізує пошук текстових файлів з розширенням .dat у поточній папці та вкладеній папці problem, дозволяє вибирати файли, переглядати їх вміст, а також отримувати та відображати таблицю з параметрами (шлях, розмір, дата створення тощо) вибраних файлів. У програмі реалізовано обробку синтаксично некоректних даних, підрахунок результатів і функцію логування ключових подій у текстовий файл.